

Kalman Filter: Milestone 4

RISC-V Vector Assembly Implementation

Deadline: 26th May

1 Purpose of this Milestone

Milestone 4 is the final stage of the Kalman Filter project. Building directly on the scalar RISC-V assembly implementations from Milestone 3, students will *vectorise* the core computational functions of both the Linear Kalman Filter (LKF) and the Extended Kalman Filter (EKF) using the **RISC-V Vector (RVV) extension**.

2 RISC-V Vector Extension (RVV)

The RISC-V Vector Extension (RVV) introduces vector registers and instructions to operate on multiple data elements simultaneously.

2.1 Vector Registers

- 32 vector registers: `v0` -- `v31`
- Configurable vector length (VL)
- Supports element-wise operations

2.2 Relevant Vector Instruction Classes (Not Limited to)

- **Configuration:** `vsetvli`, `vsetivli`, `vsetvl`.
- **Unit-stride load/store:** `vle64.v`, `vse64.v`.
- **Strided load/store:** `vlse64.v`, `vsse64.v` (required for transpose and column-wise access).
- **FP arithmetic — vector×vector:** `vfadd.vv`, `vfsub.vv`, `vfmul.vv`, `vfdiv.vv`.
- **FP arithmetic — vector×scalar:** `vfadd.vf`, `vfsub.vf`, `vfmul.vf`, `vfdiv.vf`.
- **Fused multiply-accumulate:** `vfmac.vv`, `vfmac.vf`, `vfmsac.vv`, `vfmsac.vf`.
- **Square root:** `vfsqrt.v`.

- **Reduction:** `vfredosum.vs` (ordered), `vfredosum.vs` (unordered). Use `vfredosum.vs` for reproducibility.
- **Compare and mask:** `vmflt.vf`, `vmfle.vf`, `vmflt.vv`, `vmfgt.vf`.
- **Masked merge:** `vfmerge.vfm`, `vmerge.vvm`.
- **Move:** `vfmv.v.f` (broadcast scalar to all lanes), `vfmv.f.s` (extract lane 0 to scalar FP register).

3 Problem Description

Students will vectorize the LKF and EKF implementations from Milestone 3.

The system remains:

- 23 joints
- 12 states per joint

$$N = 276, \quad M = 69$$

The objective is to accelerate:

- Matrix multiplication
- Matrix-vector operations

4 State and System Matrices

All dimensions are unchanged from Milestones 2 and 3:

$$N = 12 \times 23 = 276, \quad M = 3 \times 23 = 69 \tag{1}$$

$$\mathbf{F} \in \mathbb{R}^{276 \times 276}, \quad \mathbf{P} \in \mathbb{R}^{276 \times 276}, \quad \mathbf{H} \in \mathbb{R}^{69 \times 276}, \quad \mathbf{R} \in \mathbb{R}^{69 \times 69} \tag{2}$$

5 Implementation Requirements

- Implement:
 - `lkf_vector.s`
 - `ekf_vector.s`
- Use RVV instructions for all major computations
- Reuse scalar implementation logic where necessary
- Maintain numerical correctness

6 Memory Management

The additional consideration in Milestone 4 is **alignment**: RVV unit-stride loads (`vle64.v`) do not require alignment beyond the element size (8 bytes), but performance degrades on some implementations for unaligned accesses. Students must confirm that all pointers are passed to vectorised kernels are 8-byte aligned and discuss whether 64-byte (cache-line) alignment would further improve throughput.

7 Numerical Verification

Vectorised output must be verified against Milestone 3 scalar assembly reference. For each frame k and state component i :

$$|\hat{x}_{k,i}^{\text{vec}} - \hat{x}_{k,i}^{\text{scalar}}| \leq \epsilon_{\text{tol}} = 10^{-9} \quad (3)$$

Report the following verification tables:

1. Average absolute error per joint, averaged over all frames and state components, for both LKF and EKF.
2. Average absolute error per state component (position, velocity, acceleration, jerk), averaged over all joints and frames.
3. Maximum and minimum absolute error across the entire output for both filters.
4. A direct comparison table: Milestone 3 scalar error vs. Milestone 4 vector error for the same joint and state component, demonstrating that vectorisation did not degrade numerical accuracy.

8 Performance Analysis

A key deliverable of Milestone 4 is a **quantitative speedup analysis** comparing vectorised (M4) against scalar (M3) code. Students must report:

1. **Instruction count reduction**: Compare instruction count in M3 vs. M4 for computations that have been vectorised. Compute the reduction ratio.
2. **Speedup**: Compare the running times for implementations from M3 and M4 to report for speedup.

9 Required Analysis and Report Content

The Latex-format PDF report must include:

1. **Function-by-function walkthrough**: for each vectorised kernel, a code snippet with explanation of implementation.

2. **Numerical verification tables** as described in Section 7.
3. **Performance analysis tables and plots** as described in Section 8.

10 GitHub Repository Requirements

Push all work to branch **Milestone-4** of the existing repository. The branch must contain:

- All `.s` vectorised assembly source files.
- Makefile.
- Estimated output CSV files for vectorised LKF and EKF.

11 Evaluation Rubric (100 Points)

Component	Points
Vectorised LKF implementation	30
Vectorised EKF implementation	30
Numerical Verification	5
Performance Analysis	20
Estimated data output files	5
Report Quality and Code Explanation	10
Total	100

Table 1: Milestone-4 Evaluation Rubric